

# | Deep Learning 과 사물탐지

03.

## 전이 학습과 사물 탐지 모델

# 전이 학습과 사물 탐지 모델

## ☑ 전이 학습(Transfer Learning)

- 우리가 지금까지 해온 머신러닝과 딥러닝은 처음부터 하나씩 가르치는 개념이었다.
- 그렇기에 시간과 비용이 많이 발생한다는 단점이 있다.
- 하지만 이미 만들어진 모델에 추가 학습을 시킬 수 있다면?



- 그동안 우리가 수행한 내용은 자동차로 생각하면 수제 자동차를 만든 것이다.
- 하지만 누군가 완성도 높은 차를 만들어 놓았다면, 그냥 그것을 이용하면 된다.
- 이 완성차를 pre-trained Model 이라고 부른다.



- 또는 내가 원하는 대로 Tuning 할 수도 있다.
- 이것을 fine-tuning 이라고 부른다.

# 전이 학습과 사물 탐지 모델

## ☑ 사물 탐지 모델(Object Detection)

- 사물 탐지 모델은 사물이 어디에 있고 무엇인지 분류할 수 있는 모델이다.
- 대표적 사물 탐지 모델인 R-CNN, SSD, YOLO 는 모두 전이 학습 모델을 기본으로 삼고 있다.
- 처음부터 똑똑하게 사물을 탐지할 능력을 가지고 있다는 뜻이다.

|          | R-CNN                                            | SSD                                                | YOLO                                                  |
|----------|--------------------------------------------------|----------------------------------------------------|-------------------------------------------------------|
| 탐지 방식    | 1. 물체가 있을 법한 후보 구역 찾기<br>2. 그 구역 안의 물체 종류 맞추기    | 이미지에 다양한 크기의 격자(Grid) 를 가지고 물체 찾기                  | 이미지를 일정한 격자(Grid)로 나누고 사물 위치와 종류를 동시에 맞추기             |
| 특징       | 특정 구역을 확대해서 찾는다.                                 | 다양한 격자를 가져다 대고 찾는다                                 | 빠르게 위치와 특징을 찾는다                                       |
| 처리속도     | 느림 (실시간 처리 어려움)                                  | 빠름 (실시간 처리 가능)                                     | 압도적으로 가장 빠름<br>- 실시간 스트리밍에 최적화                        |
| 정확도      | 매우 높음                                            | 보통~높음                                              | 초기엔 낮았으나 최신 버전은 매우 높음                                 |
| 작은 물체 탐지 | 우수함<br>- 확대해서 검사하는 방식이라 잘 찾음                     | 다소 취약함<br>- 작은 물체를 종종 놓침                           | 보통~우수함<br>- 버전이 올라가면서 크게 개선됨                          |
| 장점       | - 물체의 위치를 매우 정확하게 잡아냄<br>- 작은 물체도 놓치지 않고 잘 탐지    | - R-CNN에 비해 속도가 획기적으로 빠름<br>- 다양한 크기의 물체를 안정적으로 탐지 | - 현존하는 모델 중 처리 속도가 가장 빠름<br>- 배경을 물체로 오인하는 오류가 적음     |
| 단점       | 연산량이 너무 많아 속도가 느림                                | - 작은 물체를 잡아내는 능력 취약                                | - 사물들이 너무 밀집하면 탐지가 어려움                                |
| 활용 분야    | 속도보다 <b>정확도</b> 가 생명인 분야<br>(의료 영상 분석, 위성 사진 분석) | 가벼운 사물 인식 서비스 등                                    | <b>속도</b> 와 실시간 성이 생명인 분야<br>(자율주행, 실시간 보안 감시, 로봇 비전) |

# 전이 학습과 사물 탐지 모델

## 사물 탐지 모델(Object Detection)

- 시간이 걸리더라도 정밀한 탐지를 원할 때는 R-CNN
- 빠른 탐지를 원할 때는 YOLO 가 합리적이다.

## YOLO(You Only Look Once )

- 이름처럼 한번 쳐 보고 분류/탐지가 가능한 모델
- 오픈소스 참여자들의 활발한 활동으로 가장 빠른 업데이트가 이루어지고 있다.
- 그래서 약점으로 지적되던 작은 물체 탐지 능력SSD 를 압도한다.

# 전이 학습과 사물 탐지 모델

## YOLO 설치

- yolo 환경을 설치해 보자

```
!pip install -U ultralytics
```

- Yolo 는 우리가 앞에서 배운 PyTorch 로 만들어져 있다.
- 그래서 서로 버전도 중요하다.

```
import torch

print(f"PyTorch 버전: {torch.__version__}")
print(f"GPU 사용 가능 여부: {torch.cuda.is_available()}")

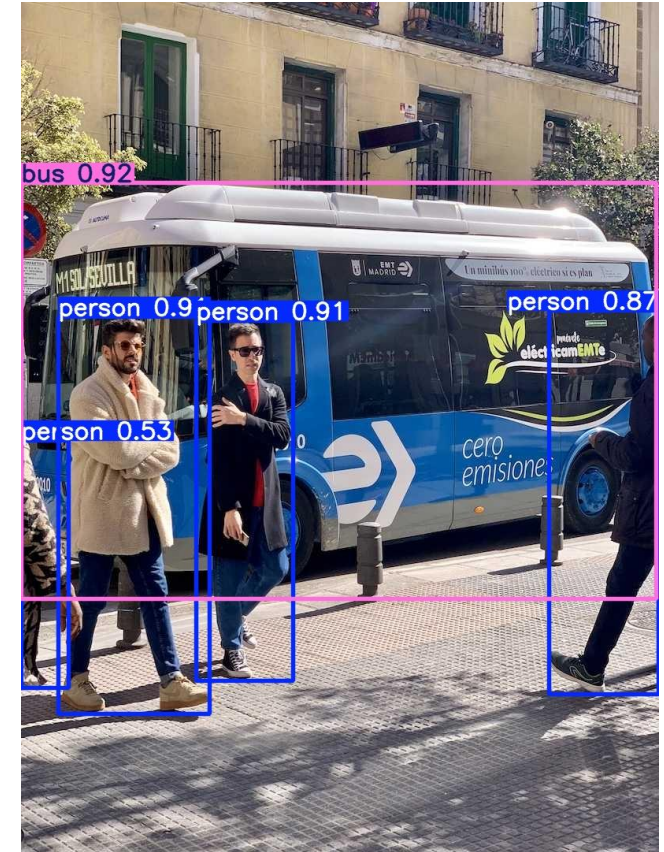
if torch.cuda.is_available():
    print(f"사용 중인 GPU 장치 이름: {torch.cuda.get_device_name(0)}")
```

06\_yolo.ipynb

# 전이 학습과 사물 탐지 모델

## ✓ YOLO 기본

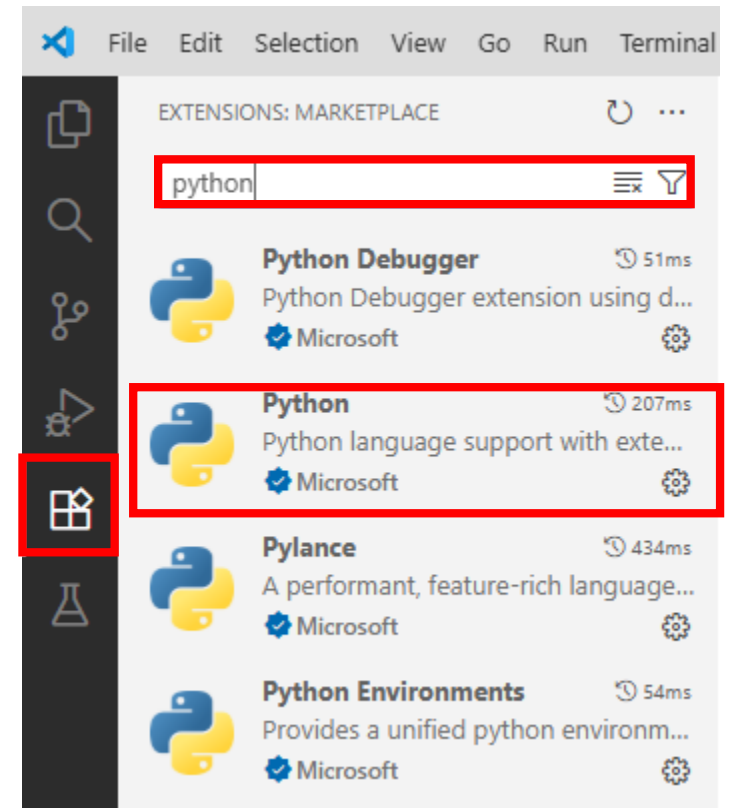
- 먼저 아래 url 을 통해 탐지할 이미지를 확인해 보자
- <https://ultralytics.com/images/bus.jpg>
- 그리고 모델을 통해 탐지/분류 하도록 해 보자



# 전이 학습과 사물 탐지 모델

## ✓ vs code 로 yolo 를 활용해 보자

- 이미 Python 과 VS code 설치 되었으므로 VS CODE 에 Plugin 을 설치
- Python 만 설치해도 연관된 plugin 들도 설치됨



# 전이 학습과 사물 탐지 모델



## vs code 로 yolo 를 활용해 보자

- Ctrl +,(Setting) 으로 이동 후 python type checking 입력
- 이후 basic 선택

python type checking

User Workspace

▽ Extensions (2)  
Pylance (2)

Python > Analysis: Type Checking Mode

Type checking modes Basic, Standard, and Strict :

| Feature                            | Basic | Standard | Strict |
|------------------------------------|-------|----------|--------|
| Variable type mismatches           | ✓     | ✓        | ✓      |
| Function return type checks        | ✓     | ✓        | ✓      |
| Type narrowing enforcement         |       | ✓        | ✓      |
| Checking of <b>Any</b> type        |       | ✓        | ✓      |
| Requires strict type compatibility |       |          | ✓      |
| Enforces complete type coverage    |       |          | ✓      |

For more details, check the [Pylance documentation](#).

basic ▾

# 전이 학습과 사물 탐지 모델

## ✓ vs code 로 yolo 를 활용해 보자

- Ctrl + shift + p 이후 settings.json 입력 UserSettings 선택



- 아래 내용 추가

```
{  
  "editor.fontSize": 24,  
  "liveServer.settings.donotShowInfoMsg": true,  
  "workbench.colorTheme": "Default Light+",  
  "python.analysis.typeCheckingMode": "basic",  
  "python.analysis.autoImportCompletions": true,  
  "editor.suggestSelection": "sprout", // 추천 설정 (가장 관련성 높은 항목을 미리 선택)  
}
```

# 전이 학습과 사물 탐지 모델

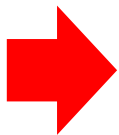
## ✓ vs code 로 yolo 를 활용해 보자

- Terminal > new terminal 선택 하여 터미널 실행
- 기본 powershell 이므로 화살표 눌러 Command Prompt 로 변경



PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\STUDY\vs\_code\01\_start> █



PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Microsoft Windows [Version 10.0.19045.2006]  
(c) Microsoft Corporation. All rights reserved.

D:\STUDY\vs\_code\01\_start> █

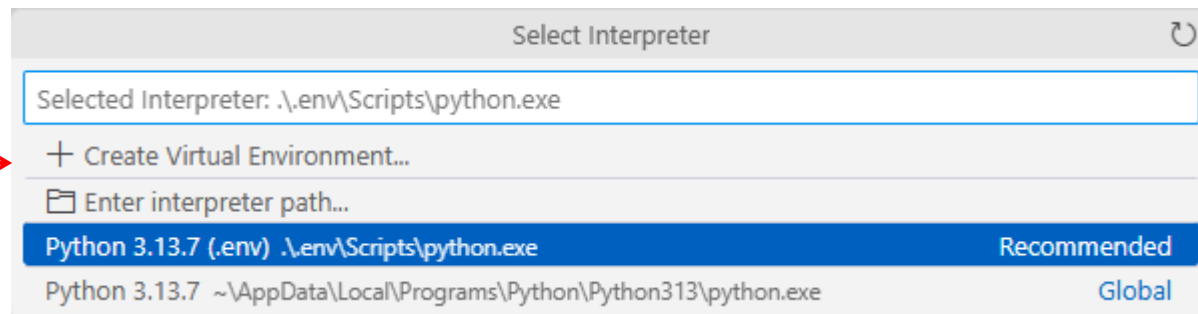
# 전이 학습과 사물 탐지 모델

## ✓ vs code 로 yolo 를 활용해 보자

- python -m venv .env(가상환경 이름) 으로 가상환경 생성 후 실행

```
python -m venv .env  
.env\Scripts\activate.bat
```

- ctrl + shift + p > Python select interpreter 선택
- 이후 가상 환경 python 선택



# 전이 학습과 사물 탐지 모델

## ✓ vs code 로 yolo 를 활용해 보자

- power shell 창을 열어주면 아래와 같이 나타난다.

```
(.env) PS D:\yolo>
```

- 그리고 필요한 라이브러리를 설치해 준다.

```
torch==2.11.0  
torchvision==0.26.0  
ultralytics  
matplotlib
```

requirements.txt

```
(.env) PS D:\yolo> pip install -r requirements.txt
```

# 전이 학습과 사물 탐지 모델

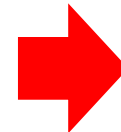
## ✓ vs code 로 yolo 를 활용해 보자

- 그리고 colab 에서 작성한 소스를 실행 해 보자
- Yolo26n.pt 는 탐지 모델로 탐지 위치와 종류를 표시한다.

```
import torch
import torchvision
from ultralytics import YOLO

model = YOLO('yolo26n.pt')
result = model.predict(
    "https://ultralytics.com/images/bus.jpg",
    save=True,
    conf=0.5,
    project="Myproject",
    name="result",
    exist_ok=True # 덮어쓰기 가능
)
```

01\_start.py



# 전이 학습과 사물 탐지 모델



## vs code 로 yolo 를 활용해 보자

- 결과가 파일로 저장되기도 하지만, 직접 결과 정보를 받아서 표현도 가능 하다.

| 속성명           | 출력된 값 (예시)                                 | 의미 및 설명                          |
|---------------|--------------------------------------------|----------------------------------|
| cls           | tensor([5.])                               | 클래스 번호(정답 인덱스)                   |
| conf          | tensor([0.9236])                           | 확신도(Confidence Score)            |
| xyxy          | tensor([[0.0, 229.36, 806.09, 757.08]])    | 사각형 박스의 네 꼭지점 절대 좌표(x1,x1,y2,y2) |
| xywh          | tensor([[403.04, 493.22, 806.09, 527.71]]) | 사각형 박스의 중심점과 크기(x,y,w,h)         |
| xyxyn / xywhn | 뒤에 n이 붙은 값들                                | 정규화(Normalized)된 좌표(0~1)         |
| orig_shape    | (1080, 810)                                | 원본 이미지의 해상도(크기)                  |
| data          | tensor([[ ... ]])                          | 위의 모든 핵심 정보(xyxy, conf, cls)     |
| id            | None                                       | 영상에서 사물 추적시 부여되는 번호              |
| is_track      | False                                      | 추적(Tracking) 상태 여부입니다.           |

02\_draw.py

# 전이 학습과 사물 탐지 모델

## Yolo 학습 시키기

- YOLO 는 이미 학습된 모델이라고 했다.
- 그럼 fashion-mnist 분류작업을 진행할 때 따로 학습을 시켜야 할까?
- YOLO 는 이미지에서 특징을 잘 찾는 법을 학습했을 뿐이기에 추가적인 학습(전이 학습) 이 필요 하다.
- YOLO 가 기본적으로 탐지 가능한 모델은 `results[0].names` 을 통해 알 수 있다.

특징을 잘 찾아내는  
모델



내가 탐지하고 싶은  
내용 학습

# 전이 학습과 사물 탐지 모델

## ☑ Yolo 학습 시키기

- fashion-mnist 를 YOLO 로 학습하여 예측 시켜보자
- 먼저 pytorch 로 부터 데이터를 받아 파일로 저장해야 한다.

```
class_names = ["T-shirt", "Trouser", "Pullover", "Dress", "Coat",  
               "Sandal", "Shirt", "Sneaker", "Bag", "Ankle-boot"]
```

```
def download_image(split='train'):  
    is_train = True if split == 'train' else False
```

```
    dataset = datasets.FashionMNIST(root='./data', train=is_train, download=True)
```

→ PyTorch 에서 데이터를 받아온다.

```
    for idx in tqdm(range(len(dataset))):
```

```
        img, label = dataset[idx]
```

```
        label_name = class_names[label]
```

```
        save_dir = os.path.join('fashion_mnist', split, label_name)
```

```
        os.makedirs(save_dir, exist_ok=True)
```

```
        img.save(os.path.join(save_dir, f'{idx}.jpg'))
```

→ 라벨명 이름이 포함된 경로를 생성, 여기에 이미지를 저장한다.

```
    print(f'{split} 이미지 저장 완료')
```

03\_train.py

# 전이 학습과 사물 탐지 모델

## Yolo 학습 시키기

- 이후 train() 함수를 통해 학습을 시킨다.
- cls 는 사물의 분류만 한다.(위치는 잡지 않음)
- runs/classify/train/weights/best.pt에 정답지를 자동 저장

```
model = YOLO('yolo26n-cls.pt')
model.train(
    data = 'fashion_mnist', # 학습할 데이터의 최상위 폴더 경로 지정
    exist_ok = True,        # 이게 없으면 result 폴더가 계속 늘어난다.
    epochs=5, # 전체 데이터를 총 5번 돌며 학습
    imgsz=28, # 이미지 크기 28x28
    batch=64, # 한 번에 처리할 미니배치 크기
    workers=4 # 데이터 로딩에 사용할 CPU 스레드 수
)
```

03\_train.py

# 전이 학습과 사물 탐지 모델

## ☑ 실시간 이미지 탐지

- YOLO 는 정지된 이미지에서 탐지도 잘 하지만 실시간 처리에 강점을 가지고 있다.
- 그래서 영상이나 웹 캠에서 넘어오는 이미지 속에 사물을 탐지하는데 훨씬 유용하다.
- 특정 영상을 이용해 자동차와 사람을 탐지해 보자

```
from ultralytics import YOLO

# 모델 로드
model = YOLO("yolo26n.pt")

# 동영상 파일 분석 및 결과 저장
# save=True를 주면 분석된 영상이 'runs/detect/predict' 폴더에 자동으로 저장됩니다.
model.predict(source="sample.mp4", show=True, save=False, exist_ok=True)
```

05\_video\_predict.py

# 전이 학습과 사물 탐지 모델

## ☑ 실시간 이미지 탐지

- 영상을 읽어오다 보면 다음 메시지가 나타난다.

Inference results will accumulate in RAM unless `stream=True` is passed, which can cause **out-of-memory** errors for large sources or long-running streams and videos.

- 영상을 통으로 읽어오면 메모리 부족 현상이 나타날 수 있어 설정으로써 프레임 단위로 가져오도록 하는 것이다.

```
results = model.predict(  
    source="sample2.mp4",  
    show=True,  
    save=False,  
    exist_ok=True,  
    imgsz=[480, 640],  
    verbose=False,  
    stream=True  
)
```

06\_video\_predict.py

# 전이 학습과 사물 탐지 모델

## 실시간 이미지 탐지

- 이번엔 웹 캠에서 사람과 사물을 탐지 해 보자

```
from ultralytics import YOLO

model = YOLO("yolo26n.pt")
# 웹캠 실행 (source=0 은 기본 웹캠을 의미)
model.predict(source=0, show=True)
```

07\_cam\_predict.py

# 전이 학습과 사물 탐지 모델

## Open CV

- 실시간 이미지 처리함에 있어 중요한 라이브러리
- 이미지 및 영상 처리를 할 수 있으며 딥 러닝을 수행 할 수도 있다.
- 우리는 이미지 표현과 카메라를 다루는 부분에만 활용할 예정이다.

```
pip install opencv-python
```

```
import cv2  
  
print(f'open CV : {cv2.__version__}')
```

08\_open\_cv.py

# 전이 학습과 사물 탐지 모델

## Open CV

- 이미지를 탐지 할 때 사용하는 함수는 predict 와 track 이 있다.
- 두 차이를 알아보자

| 기능    | predict()           | track()                  |
|-------|---------------------|--------------------------|
| 핵심 개념 | 사진 속 객체 찾기          | 영상 속 객체 따라다니기            |
| 기억력   | 이전 프레임을 기억 못 함      | 이전 프레임의 위치를 기억함          |
| 결과물   | Class (사람, 차, 의자 등) | Class + ID (사람 #1, 차 #3) |
| 화면 표시 | 박스과 라벨 이름만 표시       | 박스 위에 ID 번호가 함께 표시됨      |
| 활용 분야 | 이미지 분석, 불량품 검수      | 스포츠 전술 분석, CCTV 관제       |



# 전이 학습과 사물 탐지 모델

## ☑ 관제 시스템 만들기

- openCV 와 YOLO 를 활용 하여, 사람이 감지되면 캡처하는 모니터링 시스템을 만들어 보자

```
import cv2
from ultralytics import YOLO
from datetime import datetime
import os
```

```
cap = cv2.VideoCapture(0)
```

```
model = YOLO("yolo26n.pt")
save_dir = 'D:/img'
```

```
if not os.path.isdir(save_dir):
    os.makedirs(save_dir)
    print('폴더 생성')
```

09\_Monitoring.py

캡처 사진을 저장할 폴더 생성(없으면)

```
results = model.predict(
    frame,
    conf=0.5,
    show=False,
    verbose=False,
    classes=[0]
)
```

사람이 감지되면 캡처 저장

```
if len(results[0].boxes.xyxy.tolist())>0:
    now = datetime.now()
    formatted_now = now.strftime("%Y_%m_%d_%H%M%S")
    file_name = f"{save_dir}/{formatted_now}.jpg"
    cv2.imwrite(file_name, frame)
    print(f"저장됨: {file_name}")
```

```
if cv2.waitKey(1000) & 0xFF == ord("q"):
    break
```

1초에 한번씩 프레임을 추출

# 전이 학습과 사물 탐지 모델

## ☑ Yolo 학습 시키기2

- 앞서 우리는 fashion-mnist 를 학습시킨바가 있다.
- 이번엔 웹캠으로 mouse 이미지를 인식하여 학습 시켜보자
- 먼저 웹캠으로 마우스 이미지를 캡처 하는 프로그램을 만들어보자

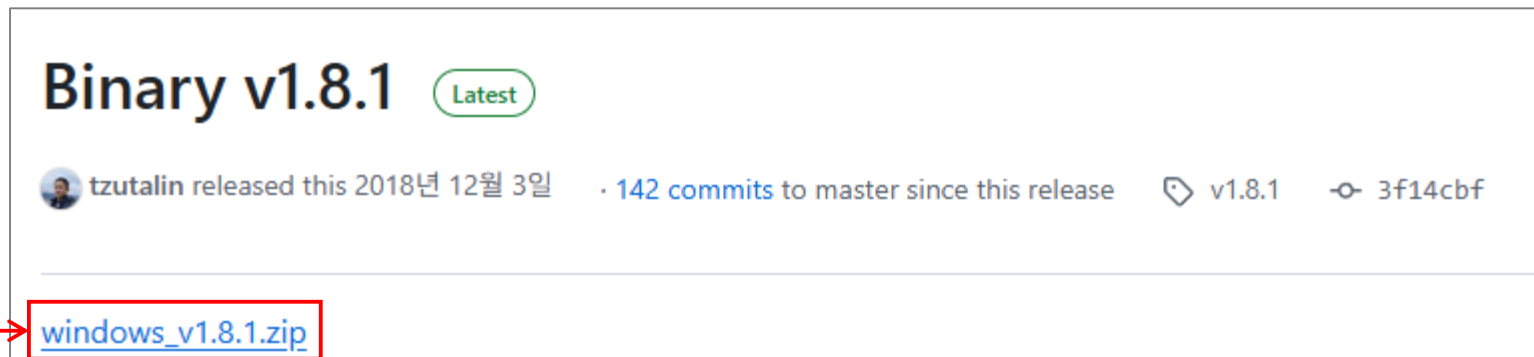
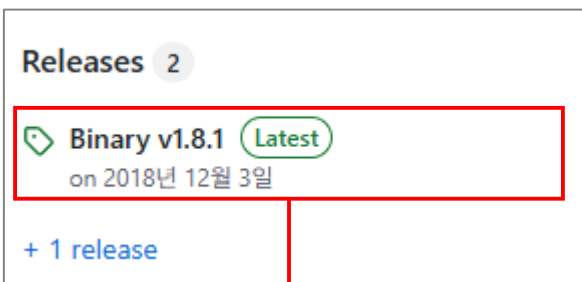


10\_capture.py

# 전이 학습과 사물 탐지 모델

## ☑ Yolo 학습 시키기2

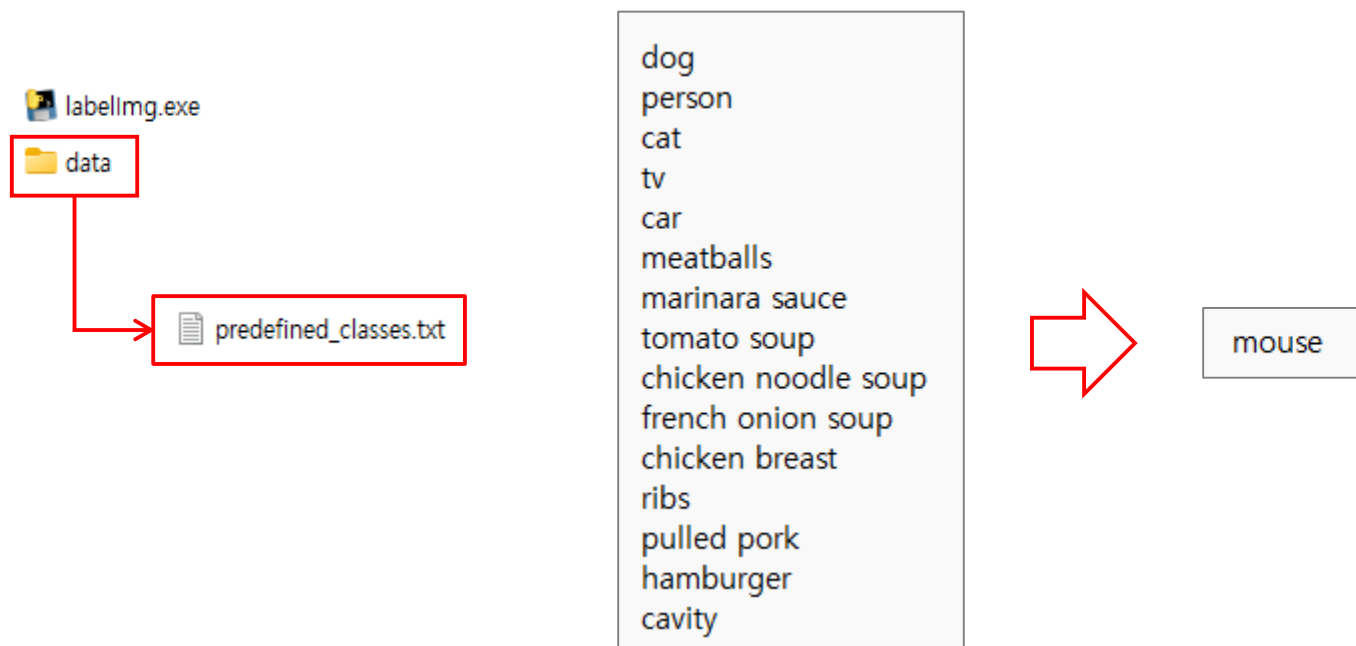
- Labellmg 설치(<https://github.com/HumanSignal/labellmg>)
- Labellmg 는 이미지 학습을 위한 라벨링을 해 주는 오픈소스 무료 프로그램이다.
- 다운로드 후 압축을 풀면 설치 완료



# 전이 학습과 사물 탐지 모델

## ☑ Yolo 학습 시키기2

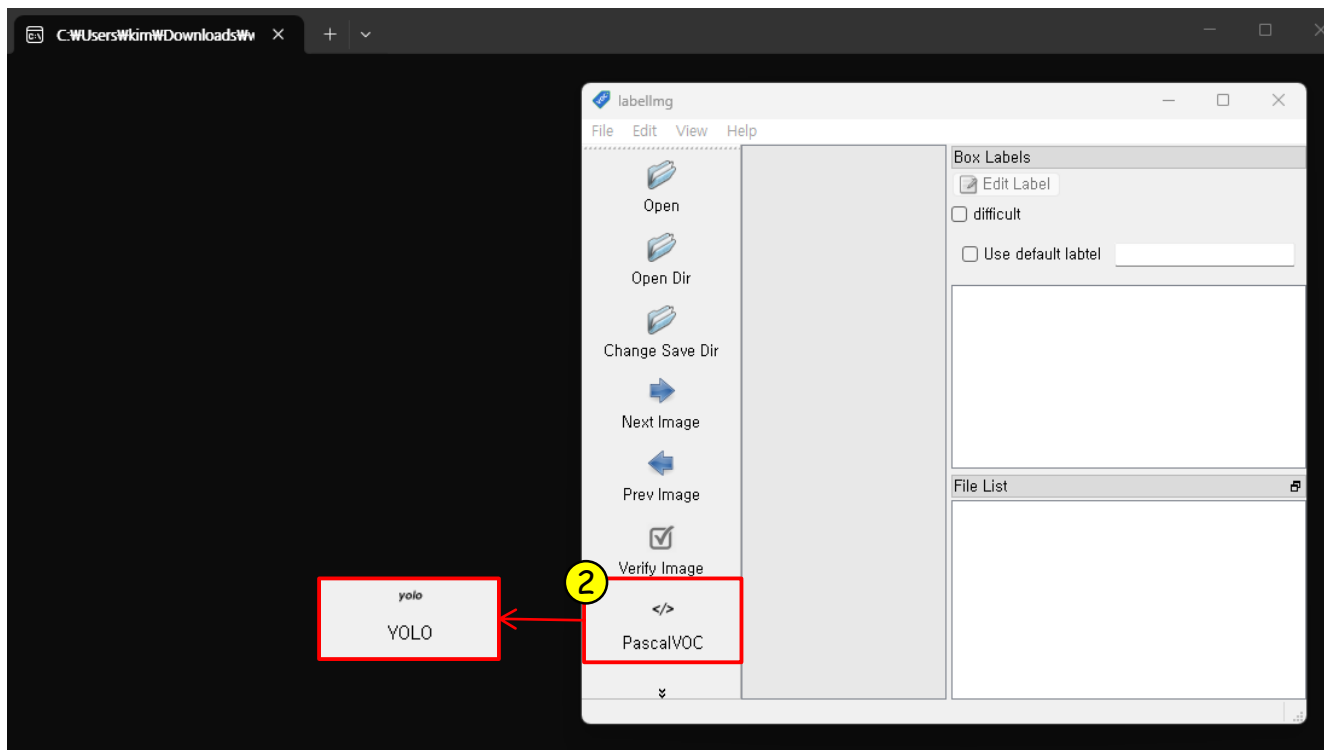
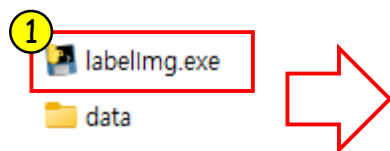
- data 폴더 의 predefined\_classes.txt 에 내용을 모두 지우고, 필요한 라벨링만 넣는다.



# 전이 학습과 사물 탐지 모델

## ☑ Yolo 학습 시키기2

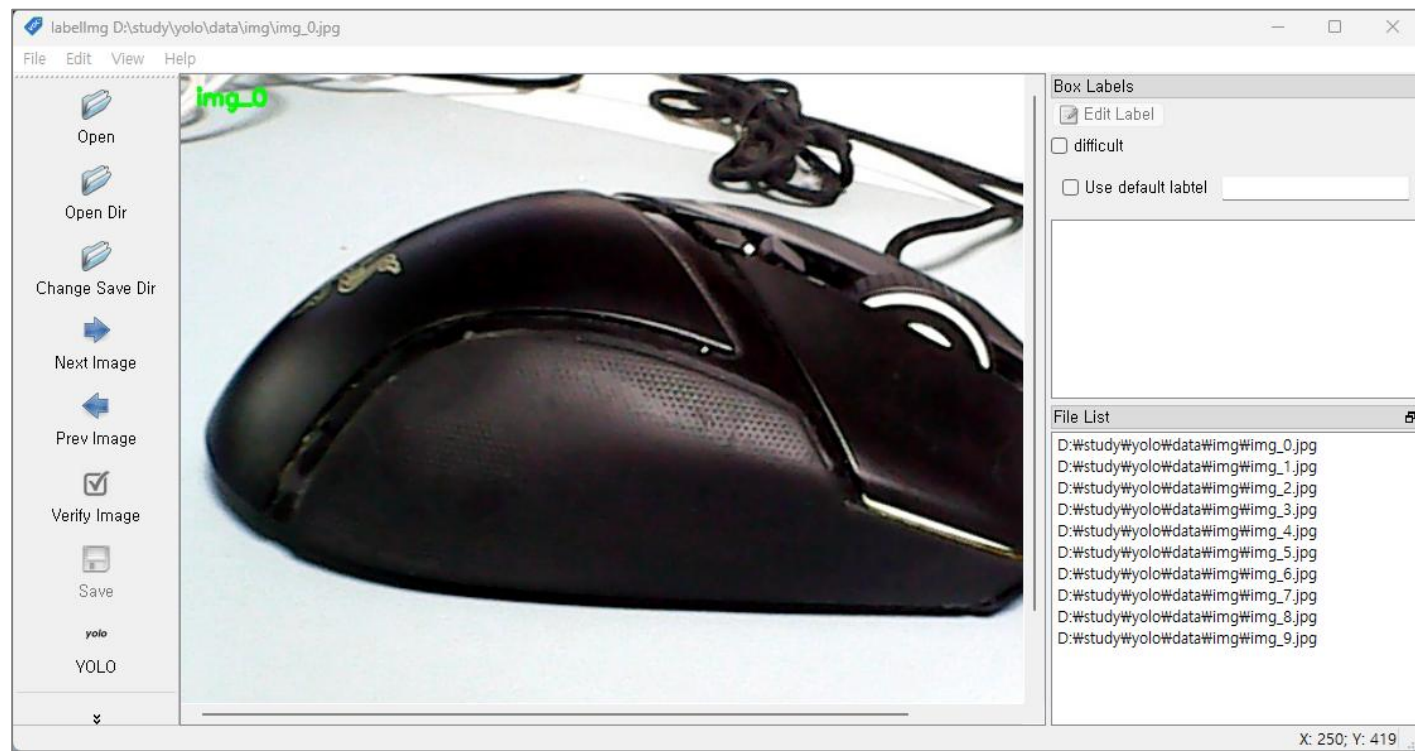
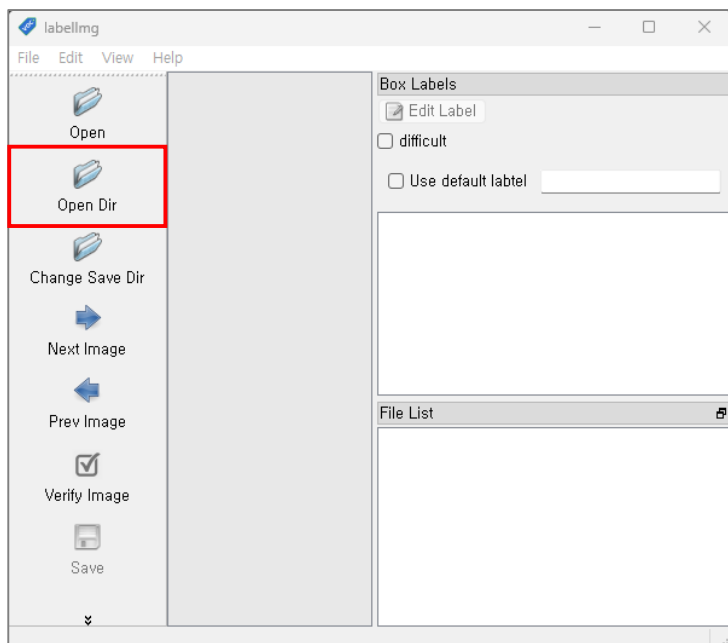
- (1) labellmg.exe 를 실행하면 커맨드창과 그래픽 창이 나타난다.(커맨드 창은 끄면 안된다.)
- (2) 저장 형태를 Pascal VOC 를 클릭해서 YOLO 로 변경



# 전이 학습과 사물 탐지 모델

## ✓ Yolo 학습 시키기2

- open dir 을 통해 학습을 위해 캡처해 둔 파일의 폴더를 지정한다.

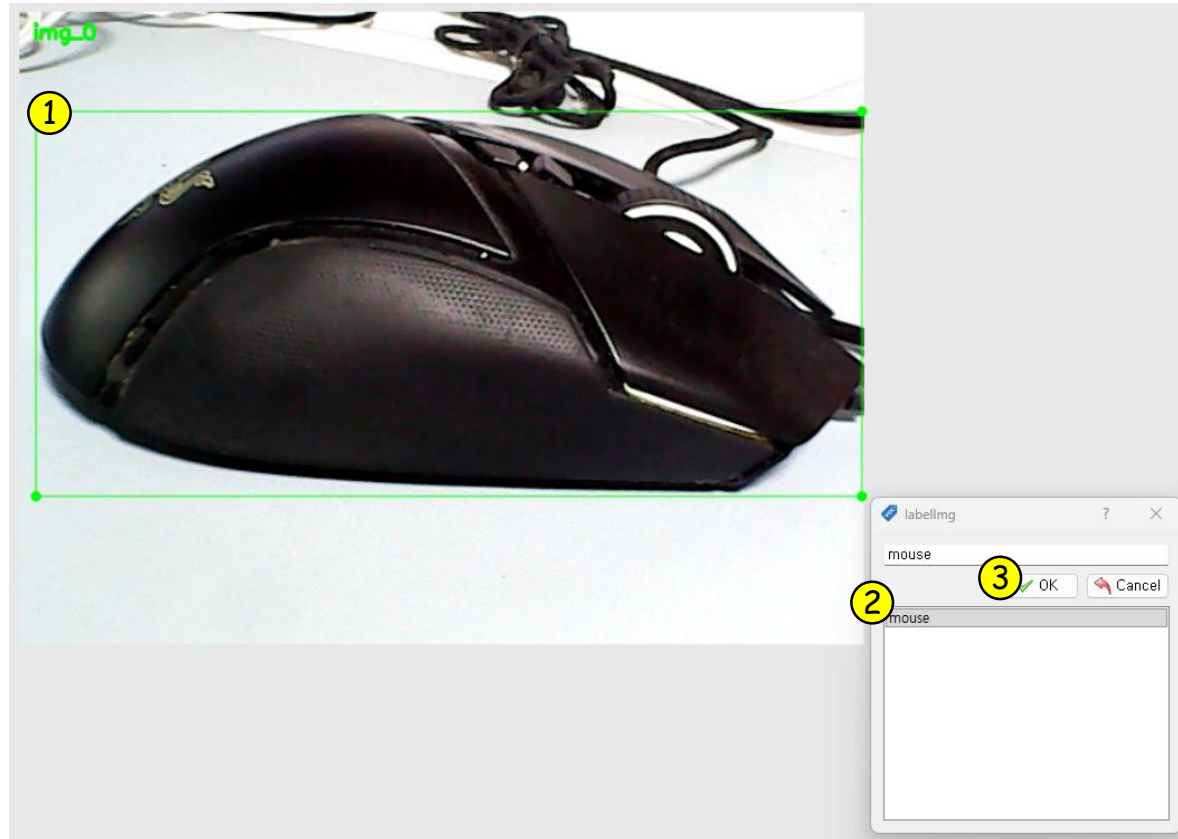


# 전이 학습과 사물 탐지 모델

## ☑ Yolo 학습 시키기2

- 다음의 단축키를 이용해 범위를 지정 후 라벨링을 선택 해 준다.

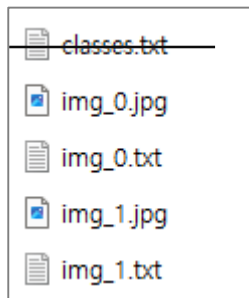
| 단축키 | 설명        |
|-----|-----------|
| W   | 이미지 범위 선택 |
| A   | 이전 이미지    |
| D   | 다음 이미지    |



# 전이 학습과 사물 탐지 모델

## ☑ Yolo 학습 시키기2

- Ctrl + S 를 눌러 저장해 준다.
- 해당 라벨 클래스 정보와, 이미지에 대한 좌표 정보가 담긴 파일이 저장 된다.
- 이때 이미지는 images 폴더로, txt 파일은 labels 로 이동 시킨다.
- 그리고 data 폴더에 data.yaml 파일을 생성해 준다.



```
# 1. 데이터셋 경로 설정
path: dataset           # 데이터셋 최상위 루트 경로
train: train/images     # 학습용 이미지 경로
val: val/images         # 검증용 이미지 경로
#test: images/test      # 테스트용 이미지 경로(선택)

# 2. 클래스 정보 설정
nc: 1                   # 전체 클래스 개수 (Number of Classes)
names:                  # 클래스 이름 목록 (라벨링할 때 지정한 ID 순서대로)
  0: mouse
```

data.yaml

# 전이 학습과 사물 탐지 모델

## ☑ Yolo 학습 시키기2

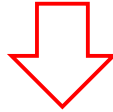
- 학습 코드를 통해 학습을 시켜주고 생성된 best.pt 파일을 루트 폴더로 이동해 주자

```
from ultralytics import YOLO

model = YOLO('yolo26n.pt')
results = model.train(
    data = 'data/data.yaml',
    exist_ok = True,
    epochs=100,
    imgsz=640
)
```

11\_label\_train.py

**ProjectRoot/runs/detect/train/weights/best.pt**



**ProjectRoot/best.pt**

# 전이 학습과 사물 탐지 모델

## Yolo 학습 시키기2

- 이제 탐지 코드를 만들어 실행 해 보자

```
results = model.predict(  
    frame,  
    conf=0.1, # 정확도 0.1 이상만 보여줘  
    imgsz=[480,640],  
    show=False, # 팝업 창 띄우기  
    verbose=False, # 프레임 로그 출력 여부  
    classes=[0,2]  
)
```

12\_test.py

인식률이 떨어진다면?

1. 이미지는 최소 300장 이상이어야 한다.
2. 다양한 각도, 다양한 거리에서 촬영해야 한다.
3. 배경도 바뀌가며 촬영 하는 것이 좋다.
4. 마우스가 아예 없는 환경도 포함해 주는 것이 좋다.
5. 학습 시 epochs=100으로 만들어주자

# 전이 학습과 사물 탐지 모델

## ☑ 데이터 셋 구하기

- 하지만 데이터 셋을 만든다는 것이 쉬운 일은 아니다.(1000 장 정도의 이미지를 라벨링 한다고?)
- 그래서 남들이 만들어놓은 데이터 셋을 활용하는 것도 한가지 방법이 될 수 있다.
- <https://universe.roboflow.com/> 에서 원하는 데이터 셋을 다운로드 받아 학습해 보자



Universe

Search 1 million+ datasets and 250k+ models...

Sign in

다운로드를 하려면 로그인 필요하다.  
Google 이나 github 로그인 가능



## Explore the Roboflow Universe

The world's largest collection of open source computer vision datasets and APIs.

1 BILLION+ IMAGES

1 MILLION+ DATASETS

250,000+ FINE-TUNED MODELS

Search datasets, models, classes...

# 전이 학습과 사물 탐지 모델

## ☑ 데이터 셋 구하기

- 원하는 검색어와 model 을 선택해서 마음에 드는 데이터 셋을 찾아보자
- 이미지 하단엔 데이터셋 이미지와 다운로드 수 등을 볼 수 있다.

Universe



computer mouse



### Search Results for computer mouse

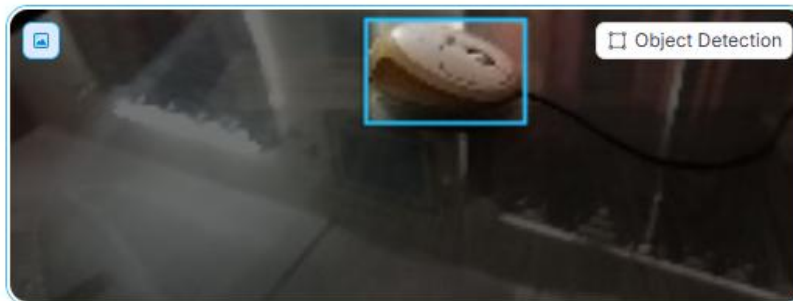
ROBOFLOW AGENT

Tell the agent what you want to build.



Identify laptop models in retail shelf photos

Open Chat →



Computer Mouse

by Machine Learning

653 images · 34

☆ 1

# 전이 학습과 사물 탐지 모델

## ☑ 데이터 셋 구하기

- Images 에서 학습에 사용된 이미지들을 볼 수 있다.
- Dataset 항목에서 데이터 셋을 구할 수도 있다.
- 사용하는 YOLO 모델을 선택 한다.

The screenshot displays the YOLO dataset management interface. At the top, there are three tabs: 'Overview', 'Images', and 'Dataset'. The 'Images' tab is currently selected and highlighted with a red box. Below the tabs, the 'Versions' section shows two dataset versions. The first version, '2024-07-12 2:22pm', is highlighted with a purple box and shows details: 'v2', '1110' images, '640x640' resolution, and a 'Fit within' button. The second version, 'custom mouse dataset', shows 'v1', '1240' images, '640x640' resolution, and a 'Fit within' button. To the right, the 'v2' version is selected, showing the date '2024-07-12 2:22pm' and a 'Download' button. Below this, the 'Popular Download Formats' section is shown, with 'YOLO26' highlighted by a red box. Other formats include YOLOv12, YOLOv11, YOLOv9, YOLOv8, YOLOv5, YOLOv7, COCO JSON, YOLO Darknet, Pascal VOC XML, TFRecord, PaliGemma, CreateML JSON, and Other Formats.

# 전이 학습과 사물 탐지 모델

## ☑ 데이터 셋 구하기

- Images 에서 학습에 사용된 이미지들을 볼 수 있다.
- Dataset 항목에서 데이터 셋을 구할 수도 있다.
- 사용하는 YOLO 모델을 선택 한다.

### Download Dataset

Use your plan's **\$60 worth of credits** for training YOLOv8, YOLOv11 and RF-DETR and more model types! Customize the dataset, preprocessing, augmentations and training.



#### Train a model with this dataset

Use this dataset as a starting point for your own custom models



#### Train from a portion of this dataset

Start training on a subset of this project or combine with other data



#### Download dataset

Get a code snippet or ZIP file

Continue



### Download

#### Image and Annotation Format

YOLO26

TXT annotations and YAML config used with YOLO26.

#### Download Options



#### Download zip to computer

Downloads all images, annotations, and classes.



#### Show download code

Custom train this dataset using the provided code snippet in a notebook.

Cancel

Continue